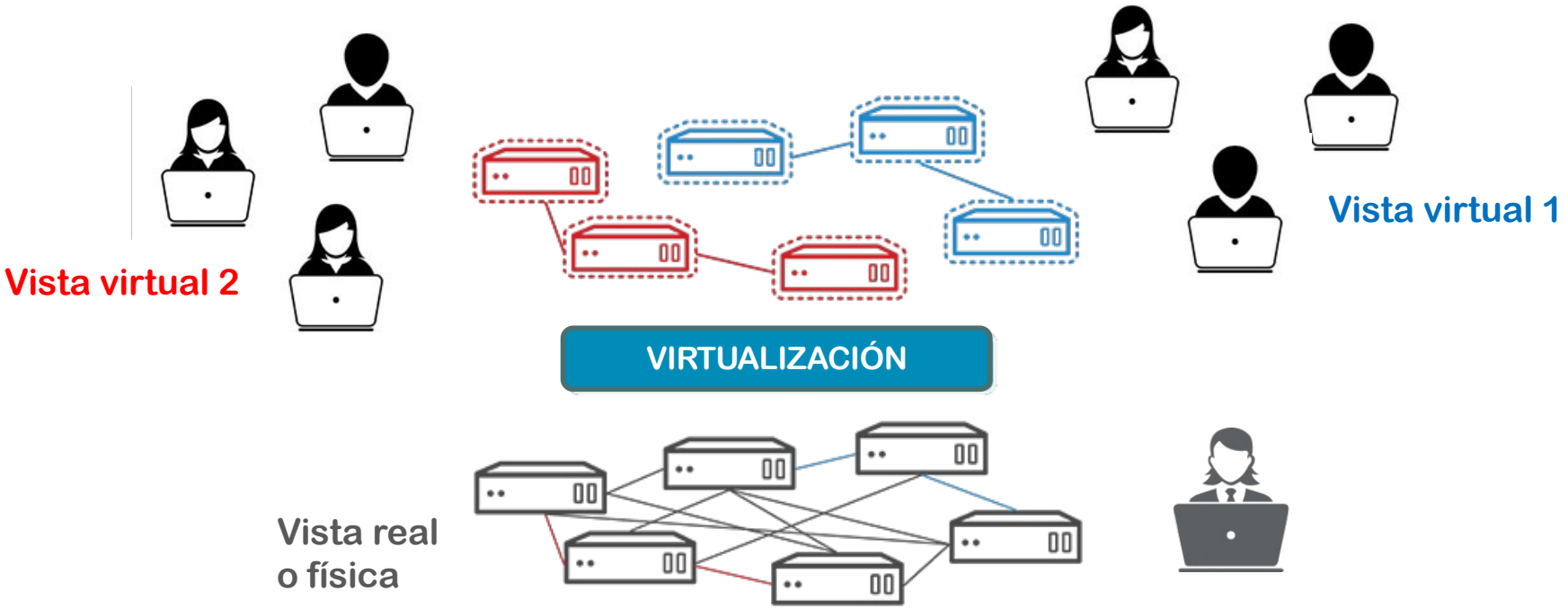
TEMA 3

Virtualización

- Definición de virtualización

La **virtualización** puede definirse como la abstracción lógica de una infraestructura física

- Proporciona una vista lógica o virtual que puede ser muy diferente del sistema físico que se abstrae
- Permite ocultar los recursos físicos reales de las aplicaciones, servicios o usuarios que los utilizan





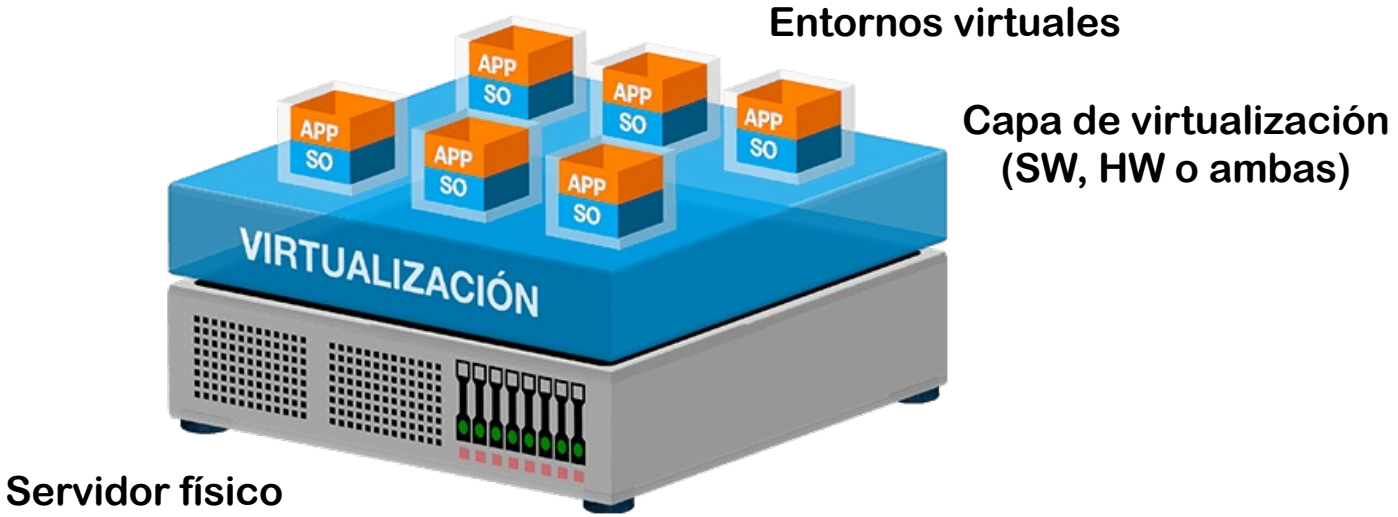
Contenidos

- **Virtualización de servidores**
- Virtualización en el continuo de computación

Virtualización

La **virtualización de servidores** permite compartir los recursos físicos de un servidor entre varios entornos virtuales de ejecución aislados entre sí

- Estos entornos virtuales reciben diferentes nombres dependiendo de la tecnología que se utilice
  - Máquina virtual (VM), servidor virtual, contenedor, etc.
- La virtualización de servidores se implementa como una capa intermedia, entre el servidor físico y los virtuales, usando técnicas SW, HW o una combinación de ambas





Virtualización

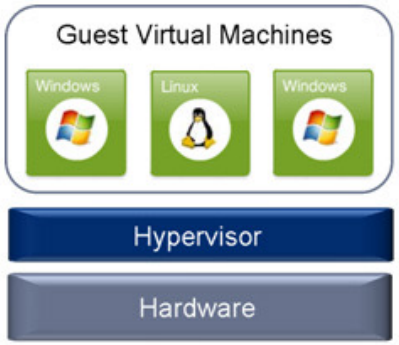
- Técnicas de virtualización de servidores
  - Existen diferentes técnicas de virtualización de servidores, las más relevantes para IoT son
    1. Virtualización con hipervisor
    2. Virtualización en el *kernel*
    3. Virtualización compartiendo el *kernel*
    4. Virtualización con unikernels

- Técnicas de virtualización de servidores

1. Virtualización con hipervisor

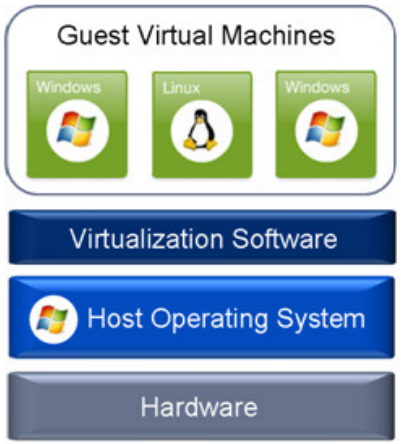
Un **hipervisor** es un componente SW que virtualiza el HW de un servidor físico y gestiona las VM que se ejecutan en él

- Hay 2 tipos de hipervisores



Tipo 1: nativo o *bare metal*

- Se ejecuta directamente sobre el HW del servidor
- Implementa las funciones básicas de un SO
- **Ejemplos**
  - Xen, VMWare ESXi, Microsoft Hiper-V



Tipo 2: *hosted*

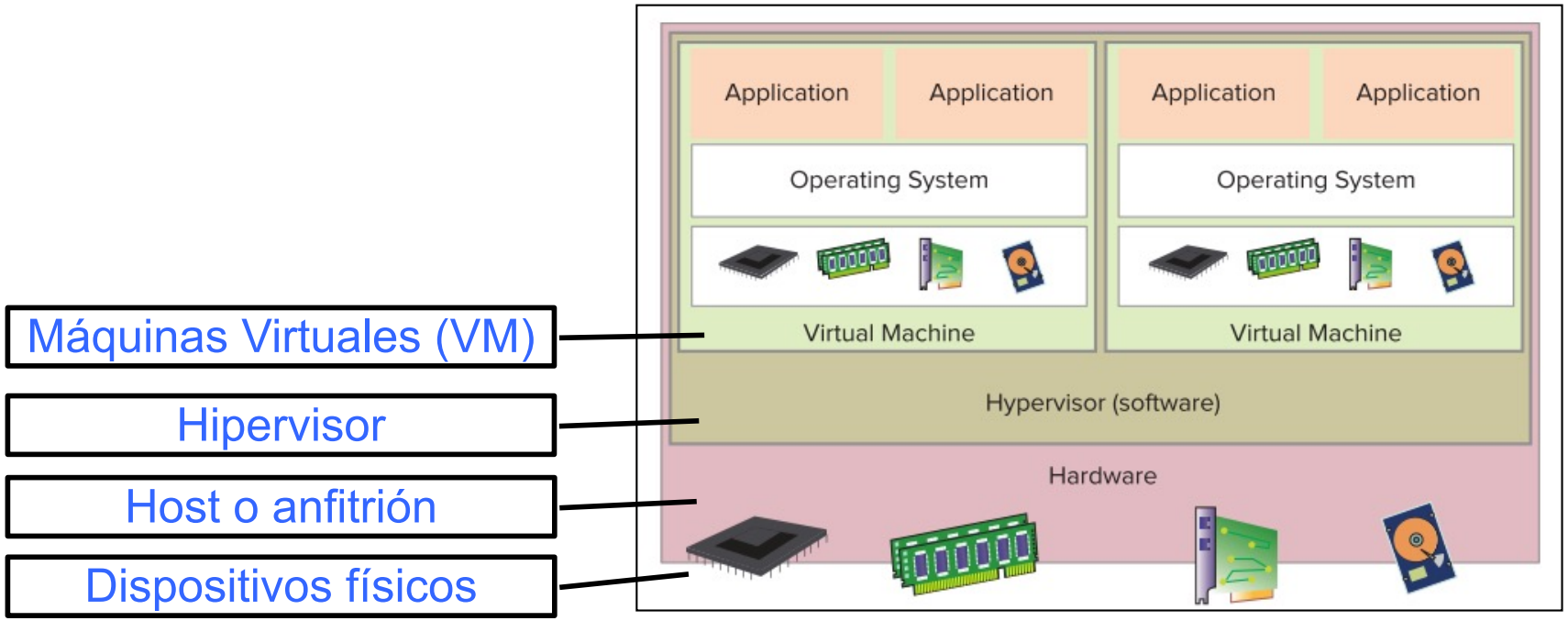
- Necesita un SO anfitrión
- Es un componente SW que añade una capa de virtualización intermedia entre el SO anfitrión y las VM
- **Ejemplos**
  - VirtualBox, KVM

- Técnicas de virtualización de servidores

1. Virtualización con hipervisor

Un **hipervisor** es un componente SW que virtualiza el HW de un servidor físico y gestiona las VM que se ejecutan en él

- Un servidor virtualizado usando un hipervisor tiene los componentes siguientes

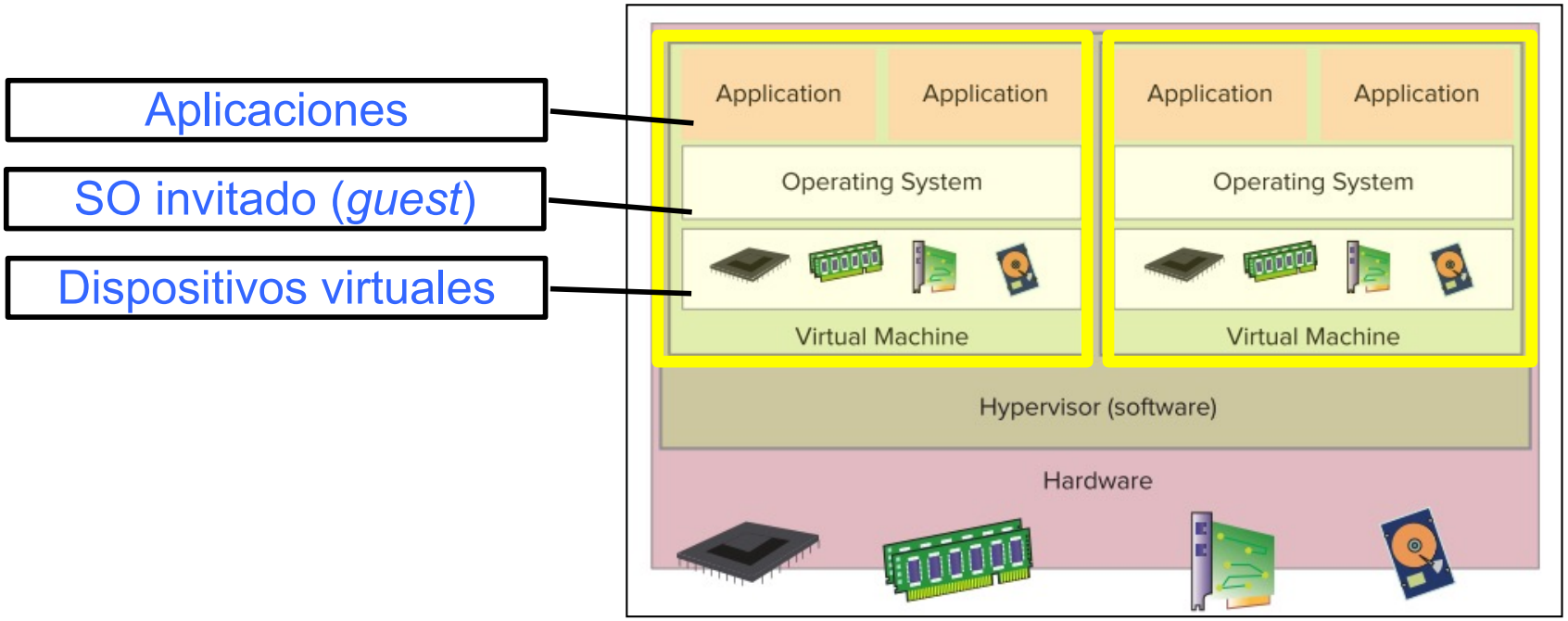


- Técnicas de virtualización de servidores

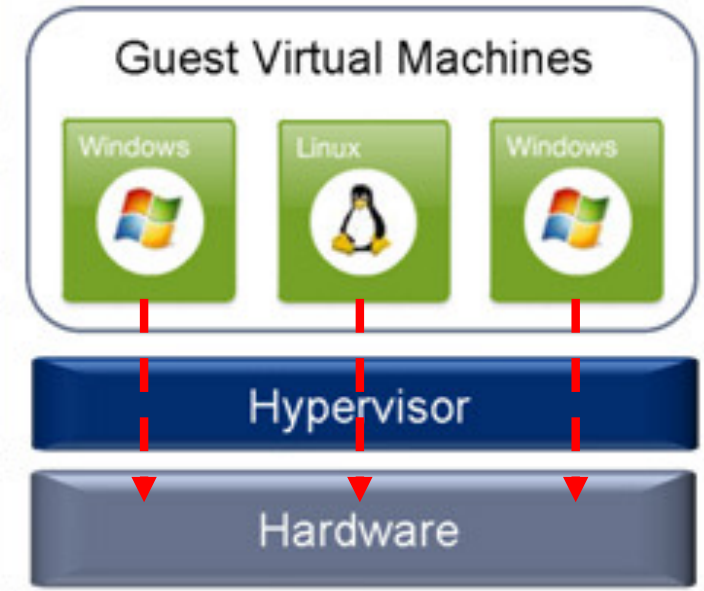
1. Virtualización con hipervisor

Una **máquina virtual** (VM) es una abstracción de los recursos HW de un servidor (CPU, memoria, almacenamiento, red) y del SO y las aplicaciones que se ejecutan en él

- Las VM ejecutadas en un mismo servidor están aisladas entre sí
- Los recursos que "ve" una VM pueden ser completamente diferentes a los disponibles en el servidor y a los de otras VM



- Técnicas de virtualización de servidores
  - **Hipervisores para la arquitectura x86**
    - En un servidor virtualizado, el hipervisor es el único que puede tener el control sobre la configuración de los recursos físicos
    - Sin embargo en la arquitectura x86 hay algunas instrucciones máquina que son problemáticas porque podrían usarse desde el SO de las VM para acceder a la configuración de los recursos físicos del servidor sin que el hipervisor pudiera evitarlo



Las instrucciones problemáticas dan acceso directo al HW del servidor desde las VM "saltándose" el control del hipervisor

- Técnicas de virtualización de servidores
  - **Hipervisores para la arquitectura x86**
    - En un servidor virtualizado, el hipervisor es el único que puede tener el control sobre la configuración de los recursos físicos
    - Sin embargo en la arquitectura x86 hay algunas instrucciones máquina que son problemáticas porque podrían usarse desde el SO de las VM para acceder a la configuración de los recursos físicos del servidor sin que el hipervisor pudiera evitarlo
    - Las técnicas que solucionan este problema se conocen como **modos de virtualización de la arquitectura x86**

Virtualización completa

- Usa técnicas de traducción binaria para detectar las instrucciones problemáticas y cambiarlas por otras no problemáticas que emulan su funcionamiento en el hipervisor
- Características
  - No necesita modificar los SO de las VM
  - Proporciona mejor aislamiento y seguridad
  - Tiene peor rendimiento
- **Ejemplo:** VMWare ESX

Paravirtualización

- El hipervisor proporciona una API a los SO de las VM. Las instrucciones problemáticas se cambian por llamadas a esa API.
- Características
  - Requiere modificar el código fuente de los SO de las VM y recompilarlos
  - Menor portabilidad (p.e. Windows)
  - Mejor rendimiento que la virtualización completa
- **Ejemplo:** Xen

- Técnicas de virtualización de servidores
  - **Hipervisores para la arquitectura x86**
    - Los primeros hipervisores introducían una penalización importante en el rendimiento del servidor, ya que emulaban por SW operaciones que, en un sistema no virtualizado, se ejecutarían directamente en el HW
    - Para mejorar su rendimiento, los fabricantes de procesadores incluyeron en sus diseños extensiones para dar soporte HW a los hipervisores
      - **Intel VT-x**
      - **AMD-V**
    - Es lo que se conoce como **virtualización nativa o asistida por HW (HVM)**
    - Con estas extensiones se elimina la sobrecarga de las otras técnicas, usando HW específico para acelerar las operaciones básicas del hipervisor
    - Además, las instrucciones problemáticas provocan ahora excepciones que son capturadas por el hipervisor → el SO de una VM no puede acceder a la configuración de los recursos físicos

- Técnicas de virtualización de servidores

2. Virtualización en el kernel

- Técnica propia de sistemas Linux que no requiere de un hipervisor
- Originariamente se usó para ejecutar, en servidores Linux, VM con diferentes versiones de Linux (Linux on Linux)
- En la actualidad, con el soporte de las extensiones HW de virtualización, es posible ejecutar también otros SO usando emuladores acelerados por HW

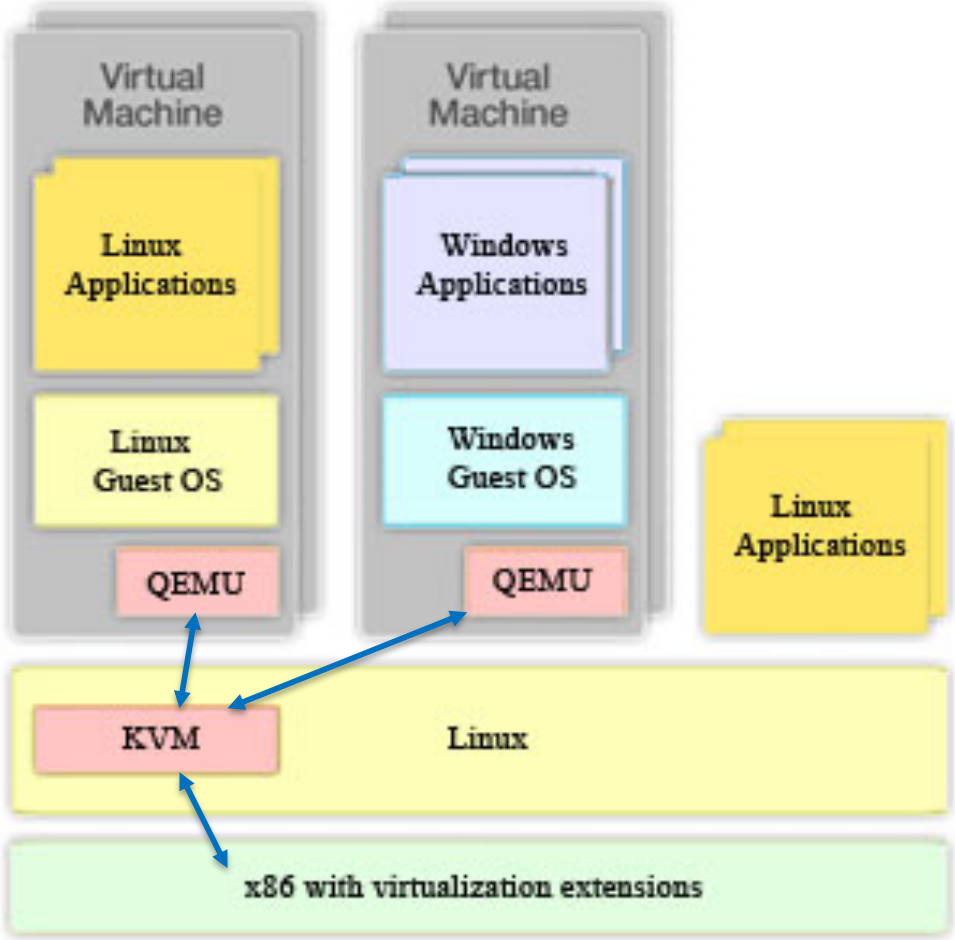
- Técnicas de virtualización de servidores

2. Virtualización en el kernel

- Ejemplo: **KVM**

KVM (*Kernel Virtual Machine*)

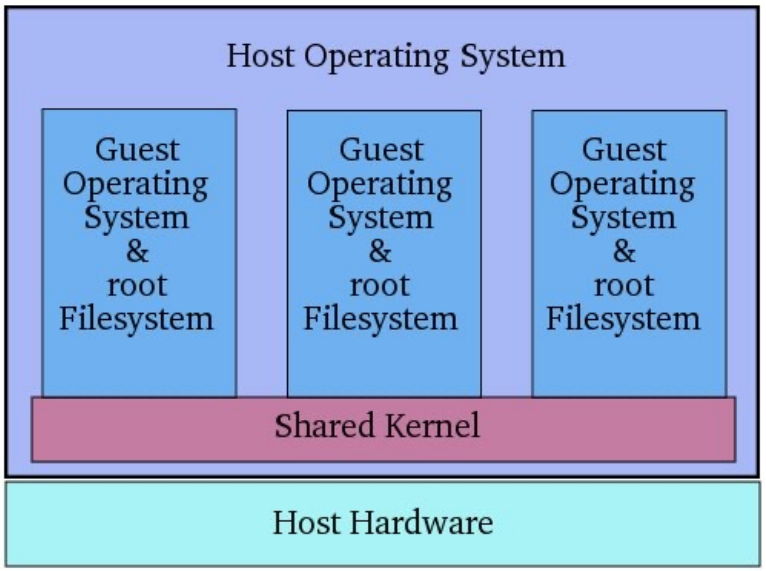
- Módulo para el *kernel* Linux que implementa la funcionalidad de un hipervisor
- Permite ejecutar VM con diferentes SO en sistemas Linux
- Usa QEMU para emular el HW de las VM
- Requiere un procesador con extensiones de virtualización para acelerar la emulación



- Técnicas de virtualización de servidores

3. Virtualización compartiendo el kernel

- Técnica originaria de sistemas Unix y Linux en la que los recursos del *kernel* del SO anfitrión se comparten entre los entornos virtuales sin necesidad de emulación HW ni hipervisor
  - El *kernel* es el que proporciona el aislamiento entre entornos y el control de los recursos
  - Cada entorno "ve" solo una versión virtual propia de los recursos disponibles
  - Los entornos reciben diferentes denominaciones, dependiendo del sistema del que se trate
    - Solaris Containers → zonas
    - FreeBSD Jails → *jails*
    - Linux VServer→ servidores personales virtuales (VPS)
    - LXC (Linux containers) → contenedores



Pros
- Es una técnica de virtualización ligera que puede ejecutar un gran número de entornos con un rendimiento próximo al nativo
- No requiere lanzar una VM con un SO completo para ejecutar un simple servicio

Contras
- Los entornos tienen que ser compatibles con el *kernel* del SO del servidor → no pueden ejecutarse a la vez diferentes SO

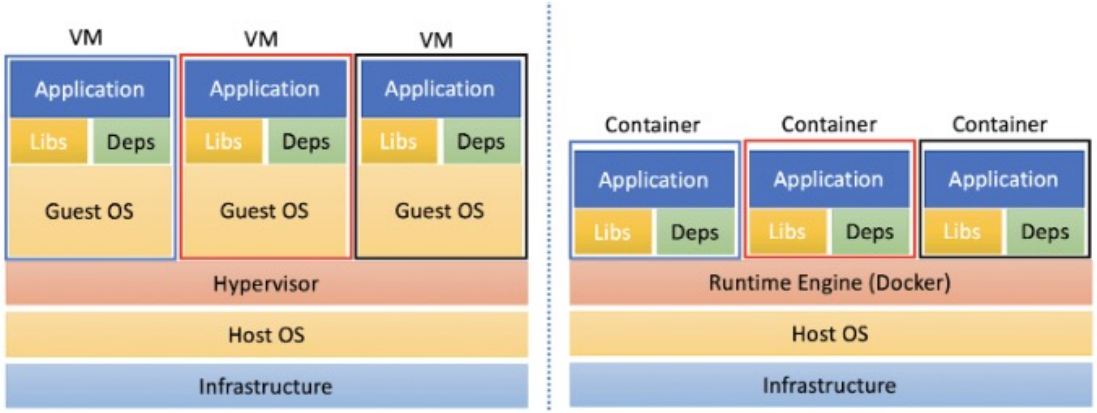
- Técnicas de virtualización de servidores

3. Virtualización compartiendo el kernel

- **Contenedores**

Un **contenedor** es un grupo de uno o más procesos de usuario que se ejecutan con recursos restringidos y aislados de otros grupos

- Los contenedores se crean usando un ***runtime*** que se encarga de comunicarse con el *kernel* para configurar y gestionar los elementos que son necesarios para ejecutar un contenedor
- La principal ventaja de los contenedores frente a las VM es que son más ligeros
  - no virtualizan el HW
  - no necesitan un SO invitado, ya que comparten el del anfitrión
- La principal desventaja es la menor seguridad, ya que los contenedores pueden ejecutarse con diferentes niveles de privilegio y comparten los recursos del SO anfitrión con otros contenedores y procesos



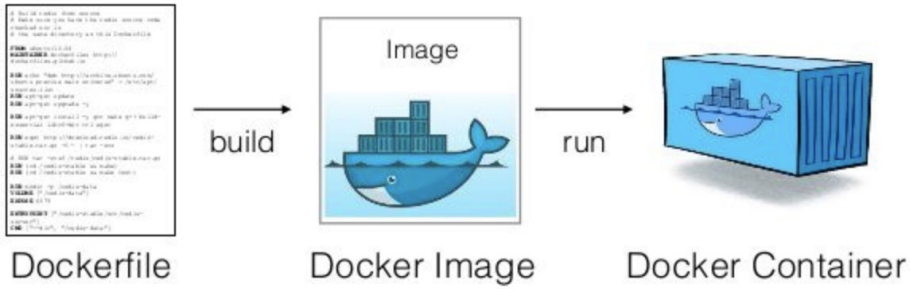
- Técnicas de virtualización de servidores

3. Virtualización compartiendo el kernel

- **Docker**
  - Es una solución basada en el uso de contenedores para la distribución y ejecución portable de aplicaciones usando imágenes

Una **imagen** es un paquete SW que contiene todo lo necesario para ejecutar una aplicación (código, *runtime*, librerías, ficheros de configuración)

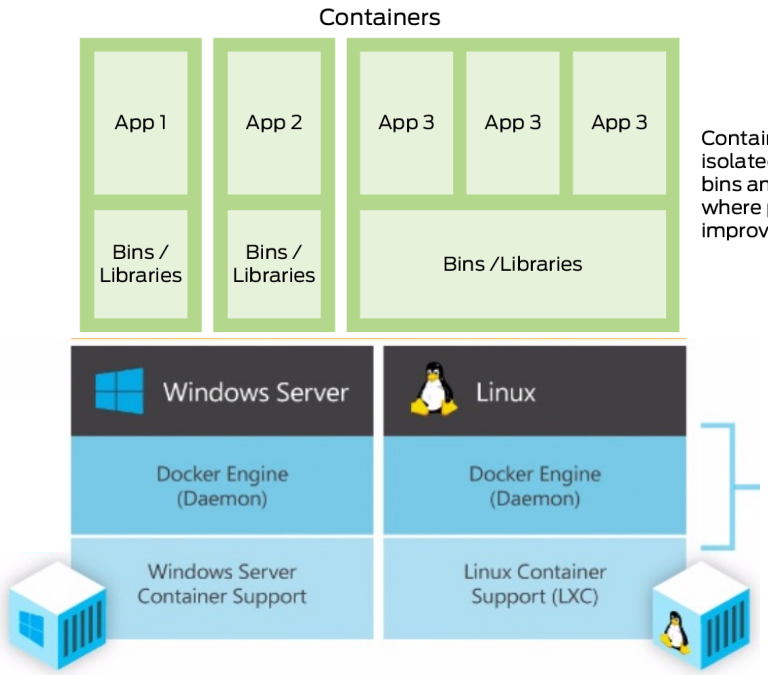
- Las imágenes se crean a partir de un fichero que contiene las instrucciones para crear la imagen (*Dockerfile*)
- Las imágenes pueden ejecutarse en cualquier sistema que tenga el motor de ejecución de contenedores de Docker instalado (*Docker Engine*)
- También pueden descargarse imágenes creadas por otros desde un repositorio de imágenes (*Docker Registry*)



- Técnicas de virtualización de servidores

3. Virtualización compartiendo el kernel

- **Docker**
  - Es una solución basada en el uso de contenedores para la distribución y ejecución portable de aplicaciones usando imágenes

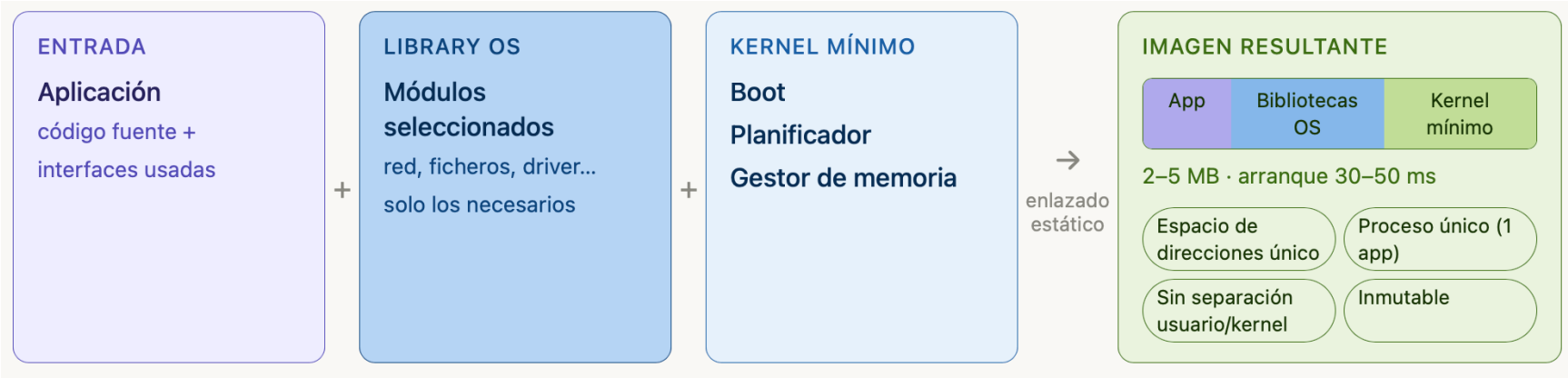


- En su origen el *Docker Engine (DE)* se implementó en Linux usando LXC (*Linux Containers*)
- Actualmente usa **containerd**, un entorno de código abierto para la gestión y ejecución de contenedores que implementa los estándares OCI (*Open Container Iniciative*)
  → la misma imagen puede ejecutarse en Linux, Windows o MacOSX
  → puede usar diferentes *runtimes*, p.e. runC, Kata, gVisor, FireCracker

- Técnicas de virtualización de servidores

4. Virtualización con Unikernels

Un **Unikernel** es un artefacto software de propósito específico (*single-purpose appliance*) que durante la compilación se especializa para dar lugar a un *kernel* autónomo y protegido contra modificaciones –inmutable– durante su despliegue (p.ej. en la nube)



- Un Unikernel se crea a partir de una aplicación, que en tiempo de compilación es *linkada* con las librerías del SO que la aplicación requiere
- El resultado es una imagen ligera, inmutable y ejecutable que solo incluye las partes del SO necesarias *para ejecutar* la aplicación
- **Ejemplos**: Unikraft, Nanos, OSv

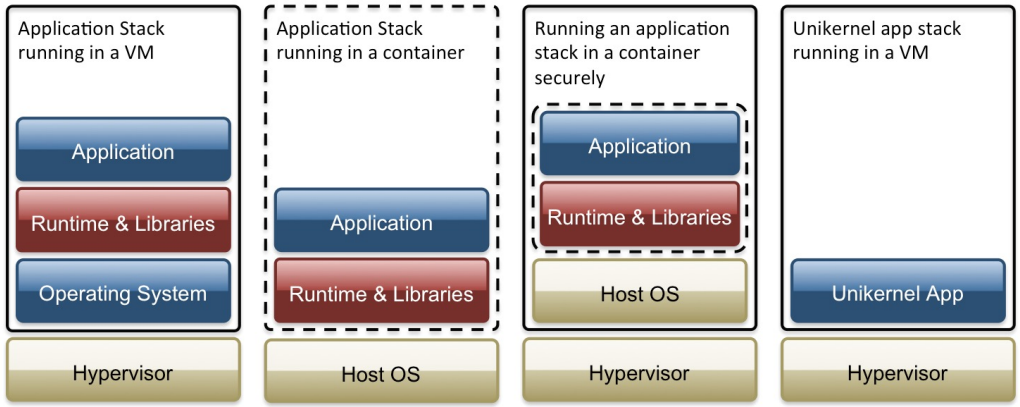
- Técnicas de virtualización de servidores

4. Virtualización con Unikernels

- Unikernel = seguridad de las VM + ligereza y rendimiento de los contenedores
  - Mejoran la seguridad comparados con los contenedores
    - Son imágenes inmutables → protección de la infraestructura sobre la que se despliegan
    - Solo incluyen lo que necesita la aplicación → superficie de ataque mínima y diferente entre ellos
  - Distribución de aplicaciones mediante imágenes ejecutables ligeras
    - Menor tamaño y tiempo de arranque que los contenedores
    - Pueden ejecutarse directamente en el HW (*bare metal*) o en un hipervisor
  - Sin embargo, también tienen desventajas
    - Dependen de la disponibilidad de SO en forma de librerías (*library* OS) con los que poder compilar
    - Menor portabilidad: hay que compilar una imagen para cada combinación arquitectura+plataforma
    - Aún no hay un ecosistema comparable al de VM y contenedores → su adopción es aún marginal

VENTAJAS	DESVENTAJAS
✓ Superficie de ataque mínima (sin shell, sin paquetes ajenos)	× El OS debe existir en forma de biblioteca
✓ Imagen ligera: 2–5 MB	× Sin multiproceso: una aplicación por imagen
✓ Arranque ultrarápido: 30–50 ms	× Ecosistema limitado — adopción en producción marginal
✓ Aislamiento similar a las VM	× Portabilidad reducida: un binario por cada combinación arquitectura + plataforma
✓ Ejecución directa en bare metal o hipervisor (Xen, KVM)	

- Técnicas de virtualización de servidores
  - **Comparación de técnicas**



	Máquina Virtual	Contenedor	Unikernel
Tiempo de instanciación	~5/10 s	~800/100 ms	<50 ms
Tiempo de arranque	Lento	Medio	Rápido
Tamaño de imagen	~1000 MB	~50 MB	<5 MB
Huella de memoria	~100 MB	~5 MB	~8 MB
Dependencia de lenguaje	No	No	Sí
Portabilidad hardware	Alta	Alta	Media
Seguridad	Alta	Media	Alta
Migración en vivo	Sí	Sí	Manual
Escenario de aplicación	Cloud computing	Cloud computing, Edge computing	Cloud computing, Edge computing, IoT edge
Multitenancy	Sí	Sí	Sí

De: https://doi.org/10.3390/pr9020351



Contenidos

- Virtualización de servidores
- **Virtualización en el continuo de computación**
  - Virtualización de red
  - Virtualización en IoT



Virtualización

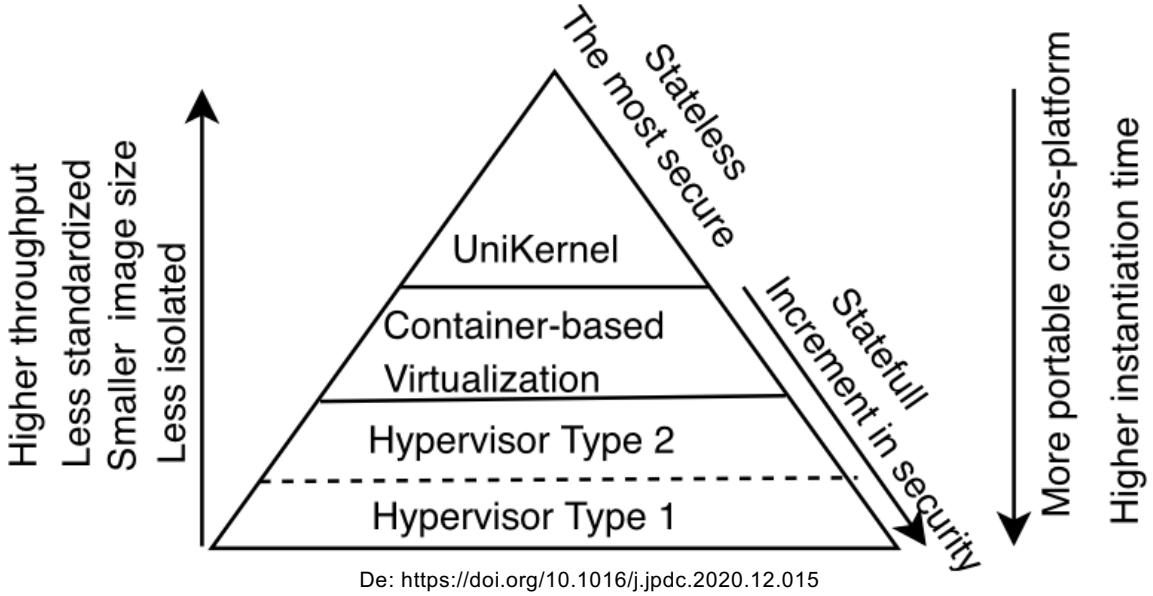
- Virtualización en el continuo de computación
  - Con el auge de la nube las tecnologías de virtualización vivieron un impulso muy importante en los últimos años por la necesidad de mejorar la eficiencia, la agilidad, el aislamiento lógico y la seguridad en el uso compartido de recursos en los CPD
  - De manera similar a lo que pasa en la nube, la virtualización está en la base de la mayoría de paradigmas en el continuo Nube-Cosa, ya que permite manejar la heterogeneidad del HW, sin embargo las técnicas de virtualización deben adaptarse a las características singulares de los nodos de borde y dispositivos IoT
    - Deben ser ligeras para poder desplegarse en dispositivos con recursos limitados
    - La movilidad de los dispositivos requiere que sean eficientes en términos de uso de memoria y tiempo de instanciación para agilizar la migración en aplicaciones con requisitos temporales estrictos
    - La naturaleza distribuida de los dispositivos y las redes de comunicación requieren gestión y control global de los recursos

Edge devices with the capability of virtualization techniques.

Edge devices type	Virtualization technique	Examples of devices
Dumb devices	NA	Sensors, actuator
Low-capability devices	Lightweight	Raspberry Pi, SBCs, Camera [107,157]
Medium-capability devices	Most lightweight	Cloudlets [204]
High-capability devices	Most heavyweight	MEC servers [72,133]

De: https://doi.org/10.1016/j.jpdc.2020.12.015

- Virtualización en el continuo de computación
  - Debido a la heterogeneidad de dispositivos y casos de uso no hay una técnica que sirva para todos los casos
    - La decisión sobre la técnica de virtualización a utilizar depende principalmente de
      - Las características de las técnicas de virtualización
      - Las capacidades de los dispositivos
      - Los requisitos de aplicación

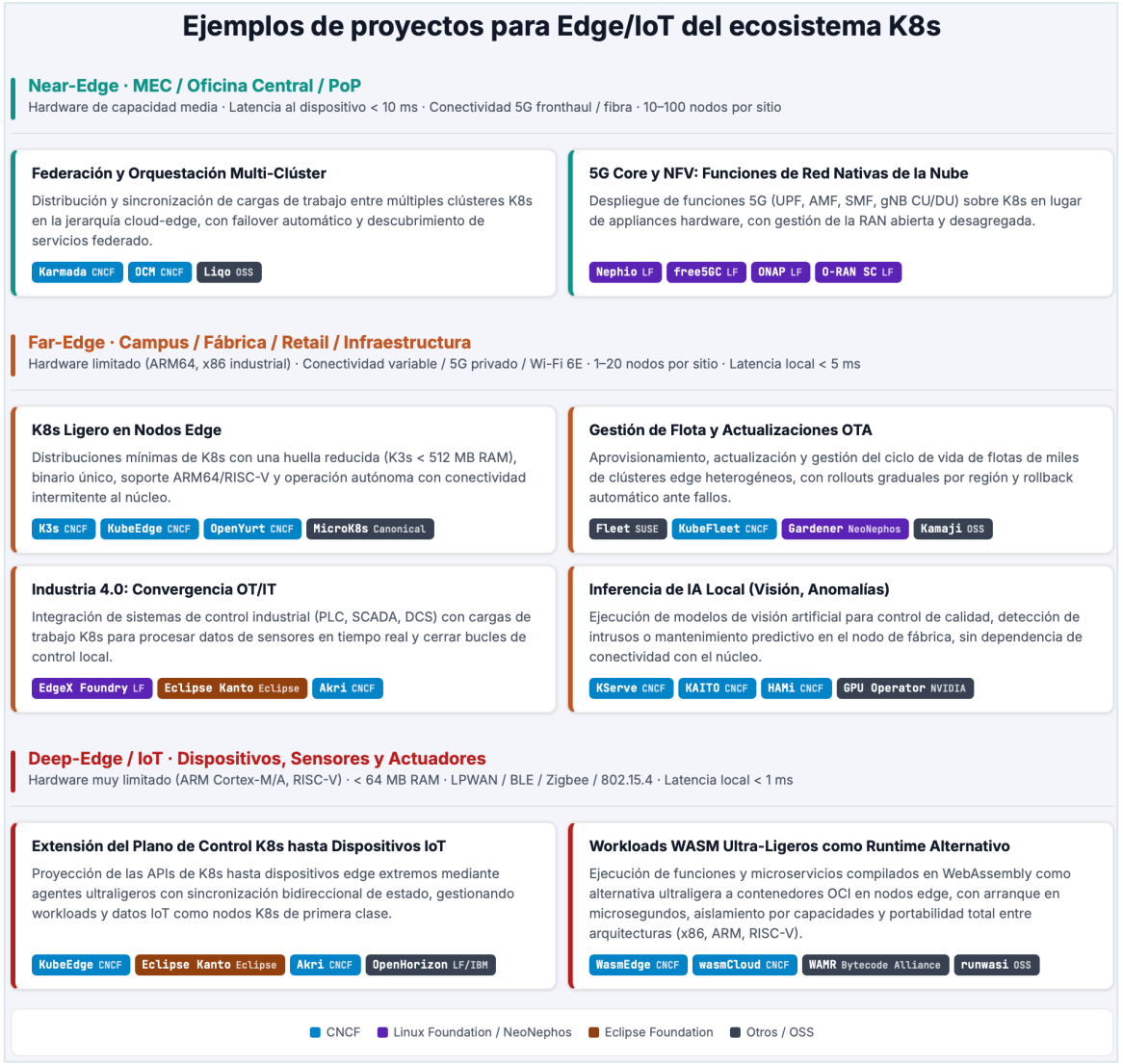


De: https://doi.org/10.1016/j.jpdc.2020.12.015

Virtualización en el continuo de computación

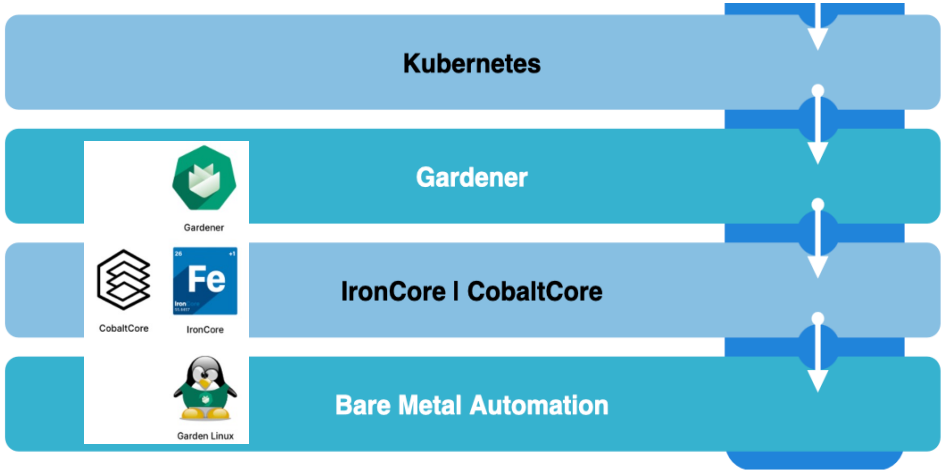


- Virtualización en el continuo de computación
  - Kubernetes (k8s)



Virtualización no continuo de computación

ApeiroRA



https://documentation.apeirora.eu/docs/architecture/

Apeiro conceptually pursues a *declarative approach* across its components, just like Kubernetes does, and adopts the idea of kubeception[1], i.e. it's using Kubernetes to run Kubernetes. As a result, most lower layers are directly run on Kubernetes.

- **Kubernetes** is the layer in Apeiro for hosting your cloud-native workloads. Note that this is vanilla Kubernetes.
- Gardener provides managed Kubernetes-as-a-Service in Apeiro across infrastructure providers. It will come with support for IronCore and CobaltCore out of the box and be extended for additional IaaS stacks.

  Gardener-managed Kubernetes nodes use Garden Linux - a small, reproducible and auditable Linux image based on Debian GNU/Linux with a focus on Linux containers and virtual machines.
- **IronCore and CobaltCore** are two infrastructure flavors of Apeiro that provide compute, network, and storage. While CobaltCore exposes an OpenStack-compatible API, IronCore comes with a declarative Kubernetes-style interface.
- Bare Metal Automation provides functionality to manage bare metal infrastructure in Apeiro through Kubernetes principles. By leveraging Baseboard Management Controllers (BMCs) and the Redfish API, it enables streamlined and automated server discovery, provisioning, and lifecycle management.

Gardener

Cloud Edge Platform | Virtualization | Management | Service Orchestration

A proven, scalable open-source system that simplifies Kubernetes cluster management across multiple infrastructures, allowing developers to focus more on innovation and less on cluster operations.

IronCore

Cloud Edge Platform | Management | Service Orchestration

IronCore is a Kubernetes-based platform that modernizes bare-metal infrastructure management by providing a unified, cloud-native interface for orchestrating compute, storage, and networking resources across distributed environments.

Relates to:

CobaltCore

CobaltCore is an opinionated OpenStack distribution that builds upon IronCore's foundation to support non-cloud-native workloads, offering enhanced features like a micro-frontend self-service portal, advanced scheduling, and seamless integration with other components.

Garden Linux

Garden Linux is a Debian GNU/Linux derivate that aims to provide small, auditable Linux images for most cloud providers and bare-metal machines.

https://neonephos.org/project_overview

- Virtualización en el continuo de computación



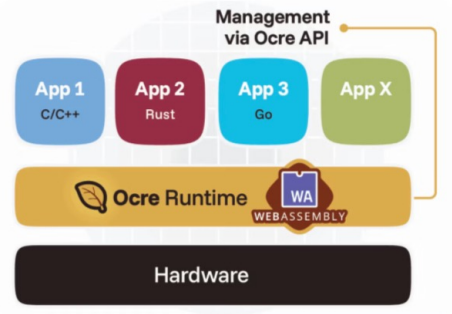
- Virtualización en el continuo de computación



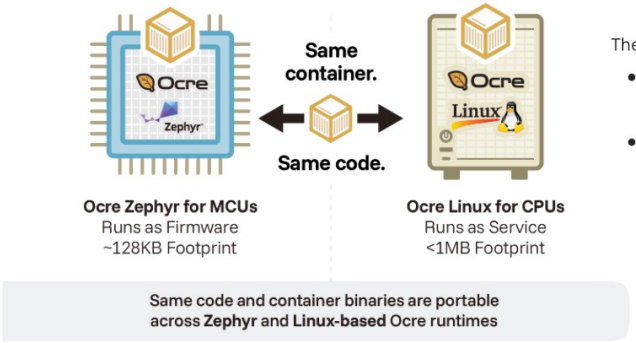
- Virtualización no continuo de computación

https://lfedge.org/projects/ocre/

Ocre leverages WebAssembly (Wasm) to extend OCI-like software containerization to resource-constrained edge devices that have historically run firmware or embedded Linux. The Ocre runtime abstracts hardware complexity and enables code written in choice of programming language to be broken out into containerized applications that are developed and managed independently and have a footprint up to 2000X lighter than traditional container binaries.

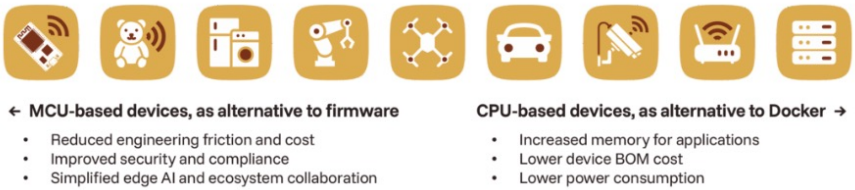


The Ocre Runtime Supports Lightweight OCI-Type Containers



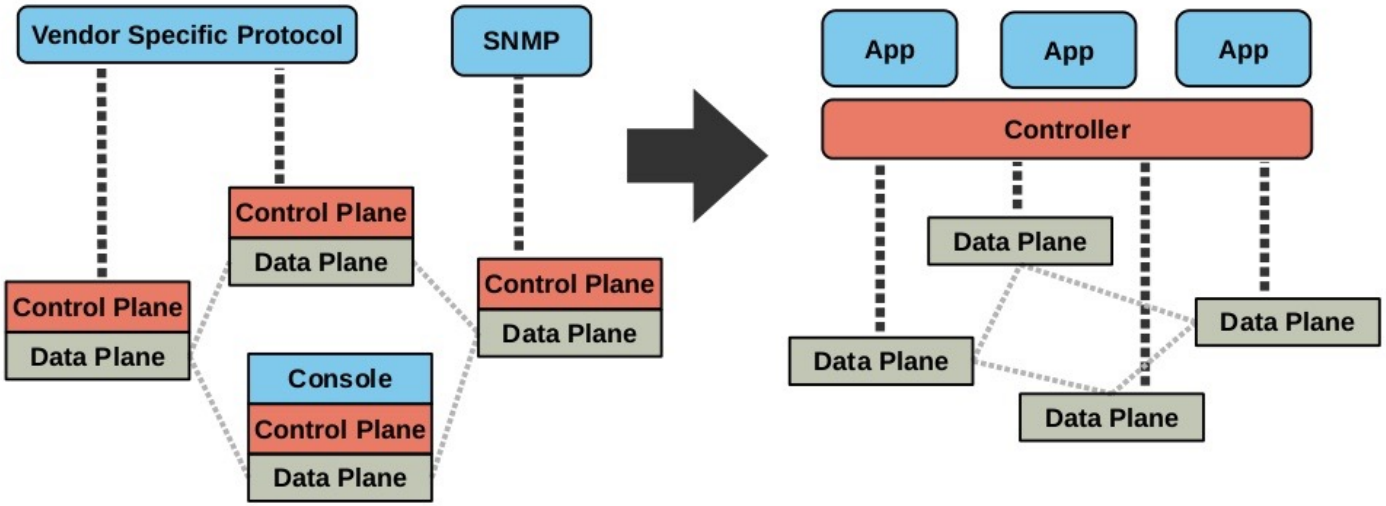
The Ocre runtime is available in two foundational versions to address a diverse landscape of resource-constrained edge hardware.

- An RTOS version for MCU-based devices with as little as 256KB of memory. The reference design includes the Zephyr RTOS and runs as firmware with containerized applications deployed on top.
- A Linux version for CPU-based devices which runs as a Linux service with a footprint of less than 1MB.



Ocre use cases span single sensors to embedded systems and general-purpose edge infrastructure

- Virtualización de red
  - **SDN (*Software-defined Networking*)**
    - Aproximación al diseño de redes que se gestionen y programen de forma centralizada para dar soporte a las demandas de escalabilidad y dinamismo de las aplicaciones actuales
    - Separa la funcionalidad de los dispositivos de red en 2 planos
      - Plano de datos, HW encargado de realizar el encaminamiento físico del tráfico de datos
      - Plano de control, parte encargada de definir las configuracionesque se aplican en el plano de datos
    - El plano de control se separa del HW, se centraliza y se expone a los servicios de red y aplicaciones a través de interfaces programáticas abiertas
    - La comunicación entre ambos planos usa protocolos abiertos

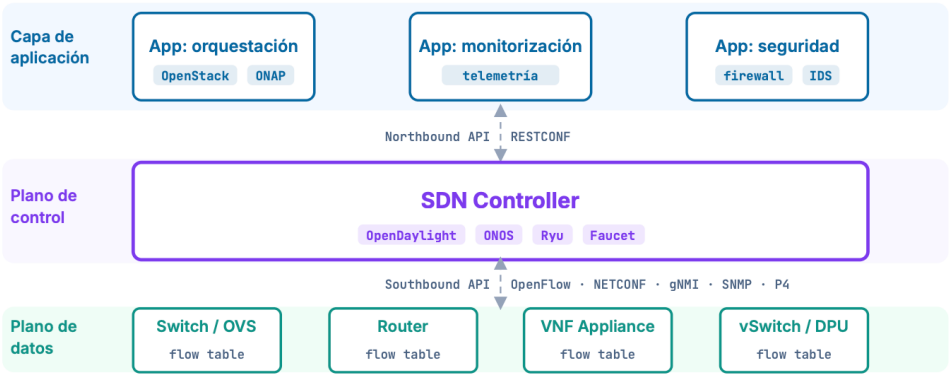


VIRTUALIZACIÓN DE RED

Redes Definidas por Software (SDN)

Arquitectura que separa el **plano de control** (lógica centralizada, programable) del **plano de datos** (reenvío en hardware genérico), conectados por **APIs abiertas**.

ARQUITECTURA SDN — 3 CAPAS



RESTCONF
REST API sobre NETCONF/YANG para aplicaciones y orquestación.

OpenFlow
Programación de tablas de flujos en switches.

NETCONF / YANG
Configuración y gestión de dispositivos de red.

gRPC / gNMI
Telemetría y configuración en streaming hacia los dispositivos.

SNMP
Protocolo clásico de supervisión y gestión de dispositivos.

P4
Lenguaje de programación del plano de datos en HW programable.

CONCEPTOS CLAVE

Controlador SDN
Software centralizado con visión global de la red. Calcula rutas y programa los dispositivos mediante APIs abiertas. Actúa como el "cerebro" de la red.

Plano de control
Lógica de encaminamiento separada del hardware; reside en el controlador SDN en lugar de en cada dispositivo por separado.

Plano de datos
Dispositivos de reenvío (switches, routers) que ejecutan instrucciones del controlador mediante tablas de flujos (match → action).

Northbound API
Interfaz entre el controlador y las aplicaciones de gestión. Suele ser REST/RESTCONF o gRPC; permite automatización y orquestación.

Southbound API
Interfaz entre el controlador y los dispositivos de red. OpenFlow es el protocolo de referencia; NETCONF/YANG es la alternativa para configuración.

Tabla de flujos
Estructura en cada dispositivo SDN que mapea reglas de coincidencia a acciones. Programada por el controlador. Puede haber múltiples tablas en pipeline.

VIRTUALIZACIÓN DE RED

Redes Definidas por Software (SDN)

Arquitectura que separa el **plano de control** (lógica centralizada, programable) del **plano de datos** (reenvío en hardware genérico), conectados por **APIs abiertas**.

VENTAJAS Y DESVENTAJAS

Ventajas

- Red programable y automatizable vía API
- La visión global facilita gestión y optimización
- Hardware genérico, menor coste
- Cambios de comportamiento sin modificar HW
- Complementa NFV: infraestructura virtualizada y programable

Desventajas

- Controlador como punto único de fallo (requiere clustering)
- Latencia adicional en las decisiones de control para flujos nuevos
- Integración compleja con redes heredadas (legacy)
- Escalabilidad del controlador en redes muy grandes o multi-dominio

CASOS DE USO Y PROYECTOS RELEVANTES

SD-WAN
Gestión centralizada de WAN sobre transporte genérico (broadband, LTE, MPLS). Separa hardware de funcionalidad; reduce costes y mejora la agilidad.

Data Center
Automatización de VLANs, balanceo de carga y microsegmentación en DC y clouds privadas.

Open vSwitch · LF | OVN · LF

Network Slicing · 5G
División de la red física en slices virtuales con QoS garantizada. Clave en las arquitecturas 5G Stand Alone (SA) y para el continuo Edge-Cloud.

ONOS · ONF

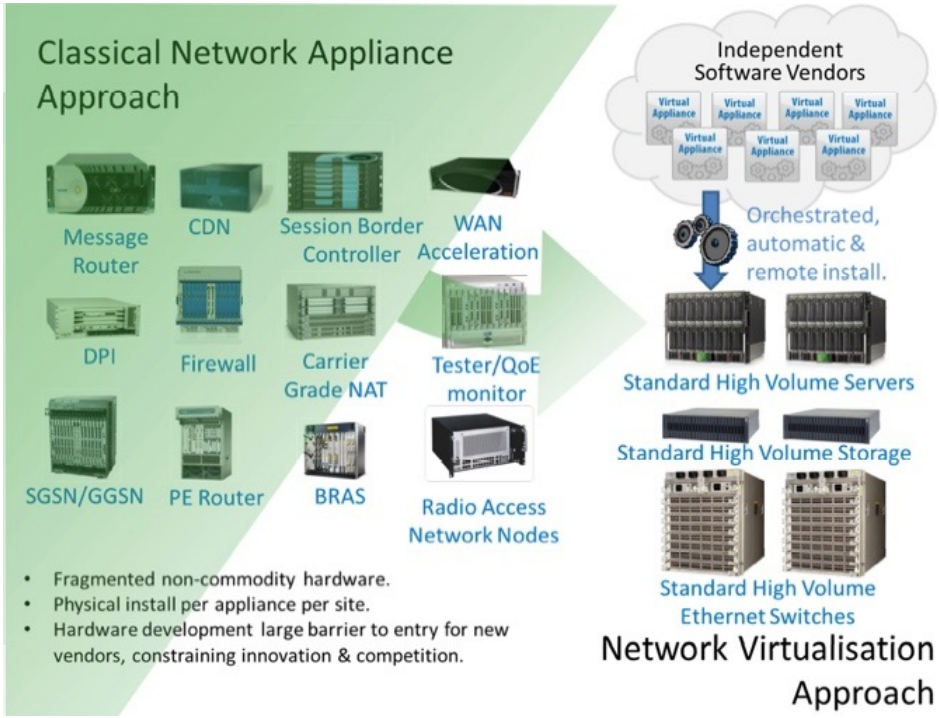
SDN + NFV
SDN proporciona la infraestructura programable que NFV necesita: el controlador actúa como VIM de red, programando OVS y los caminos de las VNF.

FD.io · LF

Seguridad dinámica
Políticas de acceso y aislamiento programables desde el controlador; reacción automática a anomalías de tráfico sin intervención manual en el hardware.

OpenDaylight · LF | Ryu · NTT | Faucet · REANNZ

- Virtualización de red
  - **NFV (*Network Function Virtualization*)**
    - La virtualización de servicios de red tiene como objetivo su despliegue como funciones de red virtuales (VNF, *Virtual Network Functions*) ejecutadas sobre HW heterogéneo estándar (p.ej. servidores, *switches*, cabinas de almacenamiento)
    - Un servicio de red podría desplegarse y migrarse de manera parecida a como se hace con una VM o cntenedor en la virtualización de servidores

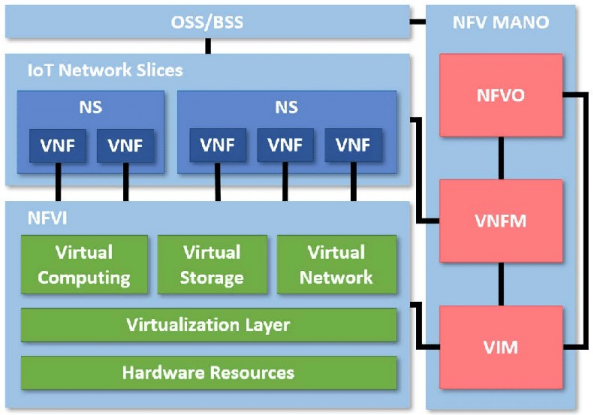


VIRTUALIZACIÓN DE RED

Network Function Virtualisation (NFV)

Enfoque arquitectónico que sustituye appliances de red propietarios (firewalls, routers, balanceadores) por software ejecutándose sobre hardware estándar de mercado. Los servicios de red se despliegan como VNF (*Virtual Network Functions*) gestionadas por una pila MANO de orquestación.

ARQUITECTURA DE REFERENCIA ETSI NFV



OSS / BSS (Operations & Business Support Systems)

Sistemas preexistentes del operador. Se comunican con el NFVO para solicitar y gestionar servicios de red.

VNF (Virtual Network Function)

Implementación software de una función de red (firewall, NAT, LB, DPI, vEPC...) desplegada como VM, contenedor o unikernel sobre la NFVI.

NFV MANO (Management and Orchestration)

Pila de gestión que cubre la orquestación de servicios de red y el ciclo de vida de las VNF.

NFVO (NFV Orchestrator)
Orquesta servicios de red y recursos entre distintos VIM.

VNFM (VNF Manager)
Gestiona el ciclo de vida de cada VNF: instanciación, escalado, actualización y terminación.

VIM (Virtualised Infrastructure Manager)
Controla los recursos de la NFVI: cómputo, almacenamiento y red (p.ej. OpenStack, k8s).

NFVI (NFV Infrastructure)

Conjunto de recursos físicos y virtuales (cómputo, almacenamiento, red) que soportan el despliegue de las VNF.

Virtualización
Capa de abstracción del HW: hipervisor, runtime de contenedores o unikernels. Presenta recursos virtuales a las VNF.

Hardware
Servidores estándar, almacenamiento y redes físicas. Aceleración de E/S con DPDK (kernel bypass) o SR-IOV (virtualización HW de NIC). Base compartida por todas las VNF.

CONCEPTOS CLAVE

Función de Red Virtual (VNF)
Implementación software de una función de red: firewall, NAT, DPI, IDS, balanceador, vEPC, vRAN... empaquetada como VM, contenedor o unikernel.

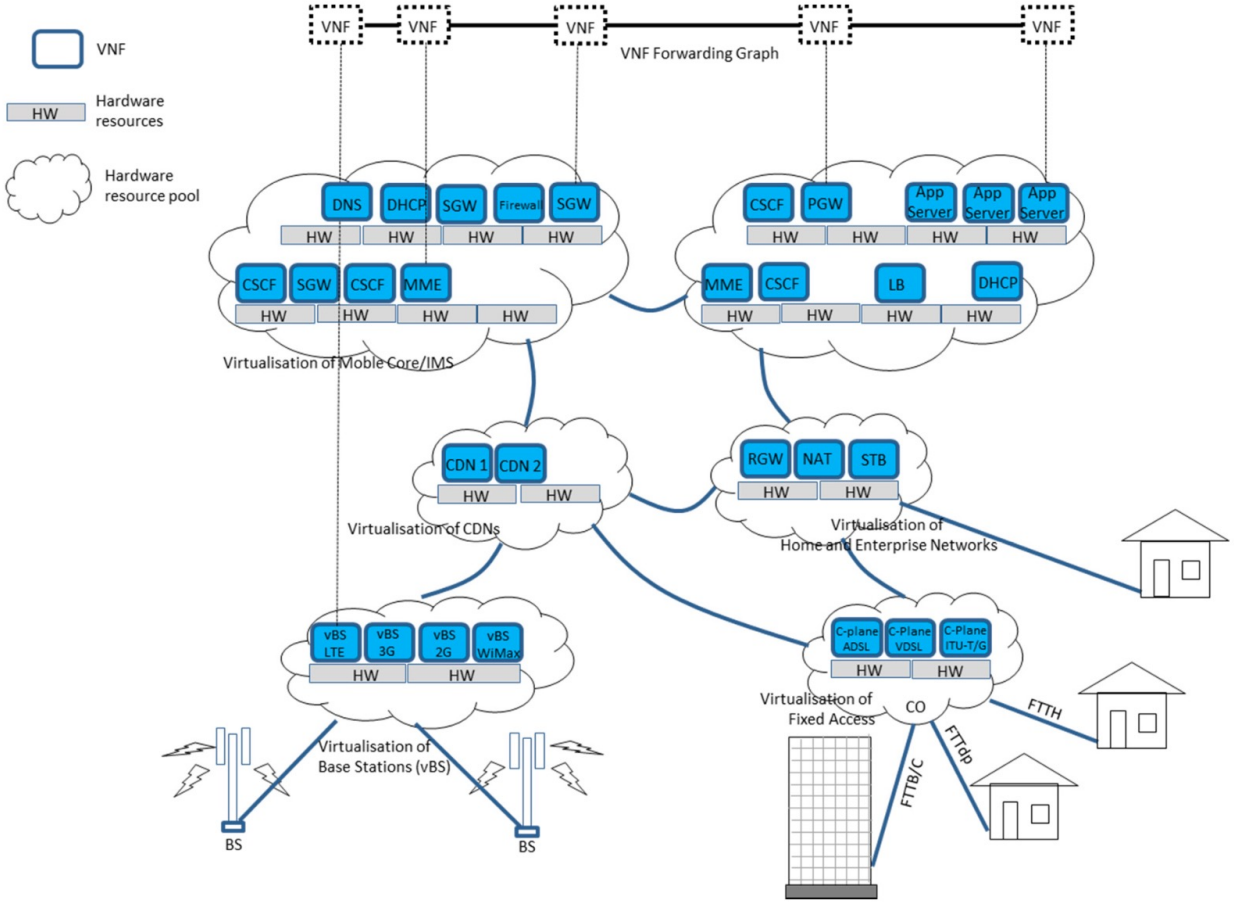
Hardware estándar
Las VNF se ejecutan sobre servidores, switches y almacenamiento estándar, eliminando appliances propietarios y su dependencia del proveedor. La NFVI soporta pooling de recursos y multitenancy sobre HW compartido.

Elasticidad y escalado
Las VNF se instancian, migran y eliminan bajo demanda. El VNFM escala horizontalmente (más instancias) o verticalmente (más recursos por instancia) según métricas de carga.

Service Function Chaining (SFC)
Composición dinámica de VNFs en grafos de encaminamiento (*forwarding graphs*) según IETF RFC 7665. El tráfico recorre la cadena de funciones en orden.

SDN + NFV
SDN controla el plano de datos (OpenFlow, P4); NFV virtualiza las funciones de red. Combinados, habilitan redes completamente programables y reconfigurables.

- Virtualización de red
  - **NFV (*Network Function Virtualization*)**
    - Los servicios se configuran dinámicamente organizando las VNF en **grafos de encaminamiento** (*forwarding graph*)



VIRTUALIZACIÓN DE RED

Network Function Virtualisation (NFV)

Enfoque arquitectónico que sustituye appliances de red propietarios (firewalls, routers, balanceadores) por software ejecutándose sobre hardware estándar de mercado. Los servicios de red se despliegan como VNF (*Virtual Network Functions*) gestionadas por una pila MANO de orquestación.

VENTAJAS E INCONVENIENTES

✓ VENTAJAS

- + **Reducción de costes** — hardware estándar frente a appliances propietarios
- + **Elasticidad** — escalar VNFs según demanda de tráfico
- + **Independencia de proveedor** — separación de SW y HW, evita lock-in
- + **Despliegue ágil** — nuevos servicios de red en horas, no en meses
- + **Reconfiguración dinámica** — SFC reprogramable sin cambios físicos
- + **Eficiencia en recursos** — consolidación, pooling y multitenancy del HW

✕ INCONVENIENTES

- – **Rendimiento E/S** — overhead de virtualización; mitigado con DPDK o SR-IOV
- – **Complejidad operacional** — pila MANO, orquestación multi-VNF, NFVI
- – **Seguridad** — mayor superficie de ataque; contenedores comparten kernel
- – **Latencia de arranque** — las VMs tardan segundos frente a hardware dedicado
- – **Interoperabilidad** — estándares ETSI maduros pero integración difícil debido a las diferencias entre implementaciones del estándar

SDN Y NFV — COMPLEMENTARIEDAD

NFV virtualiza las funciones de red (VNF) separándolas del hardware; **SDN** virtualiza la propia red que las interconecta. Juntos forman la base de las redes de nueva generación: el controlador SDN actúa como **VIM de red** dentro del NFVI, programando los switches virtuales (OVS/DPDK) y configurando los caminos de datos que las VNF necesitan. Proyectos como **ONAP** y **OpenDaylight** integran ambas tecnologías en una plataforma de orquestación unificada.

CASOS DE USO Y PROYECTOS RELEVANTES

Núcleo de red móvil (vEPC / vRAN)

Virtualización del núcleo 4G/5G: vMME, vSGW, vPGW, vUPF. Redes de acceso radio virtualizadas (O-RAN). Caso más representativo de adopción de NFV.

ONAP LF Networking · OpenAirInterface OSS · Sylva LF Networking

Edge / MEC

VNFs en nodos de acceso radio para latencia <10 ms. CDN virtualizada, cachés de contenido, procesamiento de vídeo en tiempo real.

Akraino LF Edge · EVE LF Edge

Pasarela IoT virtualizada

VNFs en gateways IoT en 3 capas: aplicación (microservicios), recursos virtualizados (Docker, SDN&NFV) y dispositivos de borde. Protocolos MQTT, CoAP, HTTP, Modbus, OPC-UA.

Eclipse Kura LF Edge · EdgeX Foundry LF Edge · Fledge LF Edge

Seguridad de red virtualizada

Cadenas de funciones de seguridad: vFirewall → vIDS/IPS → vDPI → vNAT. SFC según RFC 7665. Reconfiguración dinámica ante cambios de tráfico.

OpenDaylight SFC LF Networking · networking-sfc OpenStack

OSM ETSI · Tacker OpenStack · Nephio LF Networking

Virtualización de rede

https://sylvaproject.org

The main carriers in Europe, together with network function providers, initiated the Sylva project to address Telco and Edge use cases.

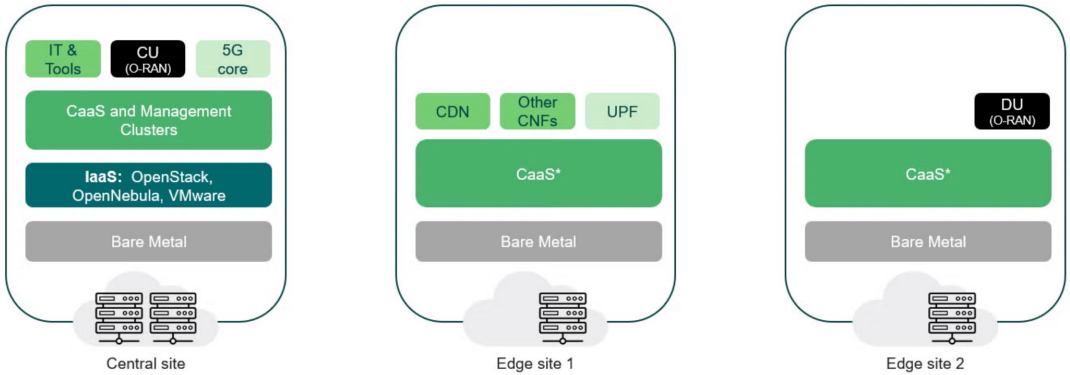
The project objectives are:

- To release a cloud software framework tailored for telco and edge requirements that address the technical challenges of the industry layer of this ecosystem
- To develop a reference implementation of the cloud software framework and create a validation program for such implementations

The Open Source releases resulting from this project will incorporate the capabilities that are required for a CaaS to address specific use cases identified by European Telcos (such as 5G, O-RAN and Edge) and will be the basis for a common infrastructure among European operators allowing the federation and integration of edge applications. Having a common framework for Operators, NF vendors will be able demonstrate, at once, that their functionality can work on such general-purpose infrastructure with the assurance that it can be deployed on any compliant platform (saving time and costs to both vendors and operators). Additionally, a common reference framework will foster digital innovation across Europe for the digital ecosystem.

Architecture

To address such use case as 5GCore Distributed UPF, CDN or Open RAN, Sylva will provide an architecture able to manage from Central to far edge site



This Cloud Native Infrastructure allows a modular approach for the deployment, interoperability, reversibility, scalability, and evolution of software components. It provides an open platform called CaaS (Container as a Service) on top of which developers can deploy applications consistently across regions and globally.

This cloud layer should align with well-established cloud native technologies and operating processes. In particular, the target architecture should be based on multi-cluster Kubernetes automated deployments on Bare Metal, managed with GitOps approach (including Infrastructure as code).

- Virtualización en IoT
  - **Técnicas de virtualización en NFV**
    - Las VNF pueden ser intensivas en cómputo o red y virtualizar funciones muy variadas
      - No hay una técnica que sea la mejor para todas las características de interés

TÉCNICAS DE VIRTUALIZACIÓN PARA VNF

	Máquina Virtual (VM)	Contenedor	Unikernel
	VNF clásica · Hipervisor	Container Network Function (CNF)	Unikernel Network Function (UNF)
Arranque	segundos	milisegundos	sub-milisegundo
Aislamiento	fuerte (hw)	moderado (ns)	total (hw directo)
SO huésped	cualquiera	kernel del host	ninguno (lib OS)
E/S	overhead moderado	casi nativo	nativo
Migración	compleja con SR-IOV	ágil	limitada
Tamaño imagen	GBs	MBs	KBs

VNF

Consumen más recursos y tardan más en desplegarse y migrar

Permite flexibilidad para ejecutar diferentes SO en la misma plataforma

CNF

Son más ligeros que las VM, lo que los hace más ágiles a la hora de desplegarlos y migrarlos

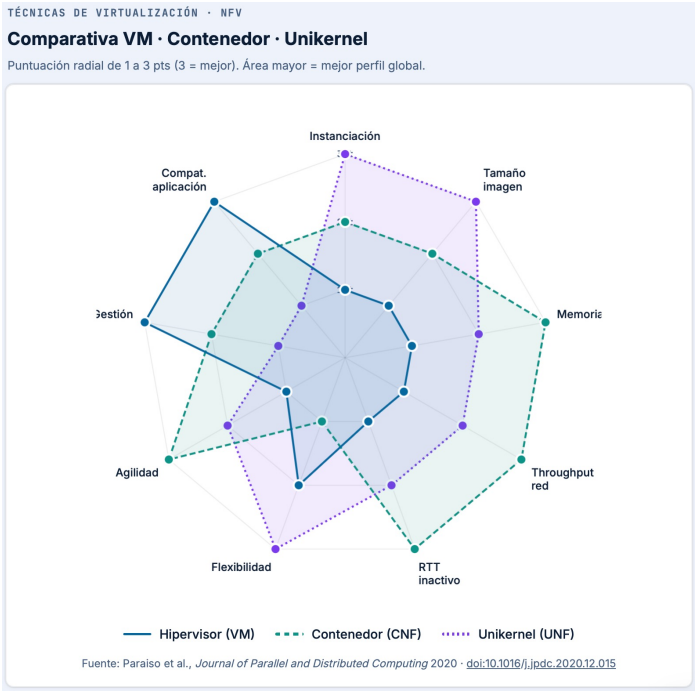
Son más restrictivos en cuanto a los SO que se pueden usar y son menos seguros

El rendimiento de E/S es cercano al nativo

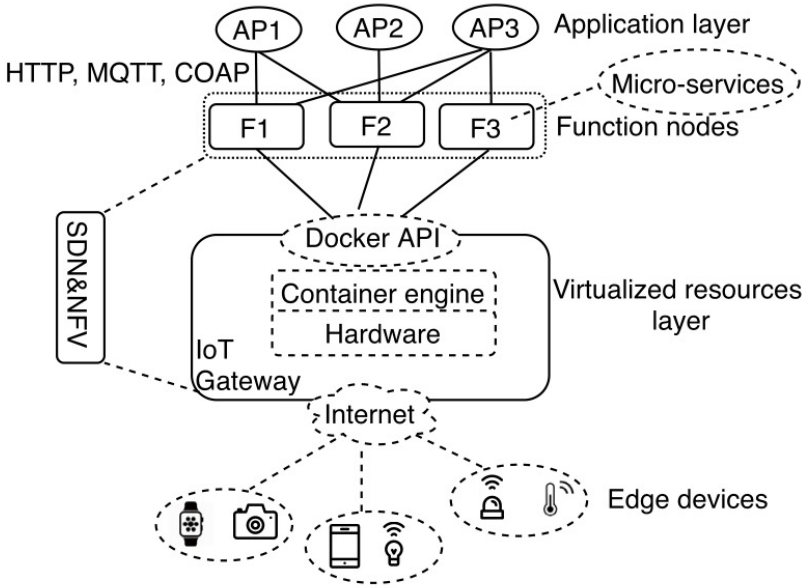
UNF

Se puedeN ejecutar directamente en HW de dispositivos con pocos recursos

Son menos portables



- Virtualización en IoT
  - **Ejemplo:** virtualización en un entorno IoT simplificado
    - Formado por 3 capas
      - **La capa de aplicación** contiene los microservicios de las aplicaciones IoT, que pueden ser compartidos
      - **La capa de recursos virtualizados** representa la pasarela IoT (IoT *gateway*) que conecta los dispositivos con la capa de aplicación
        - Usa contenedores Docker para aligerar la instanciación de microservicios y soportar la compartición de recursos
        - Proporciona servicios de aplicación: orquestador (p.ej. Kubernetes, Docker Swarm), BD y servidor web; y componentes software de virtualización de los dispositivos IoT
      - **La capa hardware** está formada por dispositivos IoT y nodos de niebla con más recursos



De: https://doi.org/10.1016/j.jpdc.2020.12.015

Principales características de la pasarela IoT

- *Escalabilidad*: permite manejar la conexión/desconexión de dispositivos IoT
- *Interoperabilidad*: permite interacciones entre diferentes aplicaciones y sistemas mediante la conversión de datos entre protocolos como HTTP, MQTT y COAP
- *Aislamiento y compartición de recursos*: el uso de una técnica de virtualización ligera facilita que las aplicaciones IoT compartan microservicios y mejora el aprovechamiento de los recursos

- Virtualización en IoT

https://lfedge.org/projects/eve/

Project EVE is building EVE-OS, a universal, open Linux-based operating system for distributed edge computing. EVE-OS aims to do for the distributed edge what Android did for mobile by creating an open foundation that simplifies development, orchestration and security of edge computing nodes deployed on-prem and in the field. Supporting Docker containers, Kubernetes clusters, virtual network functions, and virtual machines, EVE-OS provides a flexible foundation for distributed edge deployments with choice of any hardware, application and cloud.

EVE-OS can be deployed on any bare metal hardware (e.g., x86, Arm, GPU, RISC-V) or within a VM to provide consistent system and orchestration services and provides the ability to run applications in a variety of formats.

When combined with a remote controller, EVE-OS enables scalable, centralized management for large volumes of highly-distributed edge compute nodes.

